

Architettura del sistema di gestione lead e assistenza vendita

Descrizione analitica del funzionamento end-to-end: dalla cattura del lead nel CRM fino alla gestione delle obiezioni in tempo reale durante la telefonata.

COMPONENTI

Estensione browser · Lead Rework Console · Suggerimenti Vendita

AMBITO

Architettura, flussi dati, protocolli interni

INDICE

- 1. Panoramica del sistema
- 2. Cattura dati dal CRM
- 3. Lead Rework Console
- 4. Trasferimento verso Suggerimenti Vendita
- 5. Applicazione nativa Suggerimenti Vendita
- 6. Motore di ascolto e suggerimento in tempo reale
- 7. Consegna del canovaccio guida in chiamata
- 8. Miglioramento continuo della libreria
- 9. Regole di compliance e riservatezza
- 10. Disposizione delle finestre sullo schermo
- 11. Sintesi: il flusso end-to-end

01 Panoramica del sistema

Il sistema è composto da tre pezzi indipendenti, collegati tra loro esclusivamente tramite **un'estensione browser** (Chrome, Manifest V3) che fa da ponte: nessuno dei tre componenti dipende direttamente dagli altri per funzionare da solo.

COMPONENTI PRINCIPALI

CRM (Facile Salire)

Applicazione web di terze parti, non modificabile

Lead Rework Console

Artifact HTML self-contained, generazione script via AI

Suggerimenti Vendita

App nativa macOS + server locale + pagina di chiamata

Estensione browser "YesMobility CRM → Lead Rework Console"

Content script su ciascuna delle tre superfici + un service worker (`background.js`) che orchestra apertura finestre, posizionamento, passaggio dati via `chrome.storage.local`

Il resto del documento segue esattamente l'ordine in cui i dati attraversano questi componenti: dalla scheda del lead nel CRM fino al suggerimento mostrato all'operatore mentre parla al telefono.

02 Cattura dati dal CRM

2.1 Estrazione dei dati dalla pagina

Il CRM è un'applicazione basata su **Inertia.js**: ogni pagina incorpora i propri dati in un blob JSON dentro un tag `<script type="application/json" data-page="app">`, già presente nel DOM al caricamento — non serve interrogare API separate. Il content script `crm-content.js`, iniettato solo sulle pagine di dettaglio lead (`https://app.facilesalire.it/leads/*`), legge quel blob una sola volta e ne estrae i campi rilevanti (anagrafica, note, storico attività, stato/fase).

Un pulsante fluttuante, aggiunto dal content script direttamente nella pagina del CRM, avvia il trasferimento al click dell'operatore.

2.2 Trasporto dei dati e rilevamento pagina obsoleta

I dati estratti vengono scritti in `chrome.storage.local` (chiave `pendingLead`) — l'unico canale di trasporto usato in tutto il sistema per passare dati tra pagine/processi diversi dello stesso browser, dato che i content script vivono in "mondi isolati" e non condividono variabili JavaScript tra loro. Contestualmente viene inviato un messaggio `{"type": "leadCaptured"}` al service worker.

Poiché il CRM è una SPA, l'URL della scheda può restare invariato mentre l'utente naviga tra lead diversi: per evitare di catturare dati di un lead diverso da quello visualizzato, il content script confronta l'ID lead nei dati estratti con quello atteso dall'URL corrente e, in caso di mismatch, mostra un avviso esplicito invece di procedere silenziosamente con dati sbagliati.

2.3 Apertura della Lead Rework Console

Ricevuto `leadCaptured`, `background.js` cerca una scheda già aperta della console (`chrome.tabs.query` su `file:///*/lead-rework-console.html`):

- **Scheda già aperta** → viene ricaricata (garantisce sempre l'ultima versione del file) e riposizionata nella colonna di schermo dedicata; i dati in attesa vengono letti da soli al ricaricamento.
- **Nessuna scheda aperta** → ne viene creata una nuova (`chrome.windows.create`, `type: "popup"`), senza barra indirizzi/tab), posizionata e dimensionata dinamicamente in base alla risoluzione reale dello schermo (vedi §10).

Il content script `console-content.js`, iniettato nella pagina della console, consegna il lead in attesa alla pagina vera e propria tramite `window.postMessage` (il mondo isolato del content script non può chiamare direttamente le funzioni JavaScript della pagina).

03Lead Rework Console

3.1 Architettura dell'artifact

Un singolo file HTML self-contained (nessuna build, nessuna dipendenza server): CSS, JavaScript e dati (profilo azienda, libreria script, stati lead) sono tutti inline. Lo stato dell'applicazione vive in un oggetto `state` globale; ogni modifica passa da una funzione `render()` che ridisegna l'interfaccia — pattern manuale, senza framework.

3.2 Import dei dati e estrazione assistita

Oltre all'import automatico dal CRM (§2), i dati possono arrivare da inserimento manuale o da file caricati (screenshot, PDF convertiti pagina-per-pagina in immagini, CSV/Excel via SheetJS). Un secondo passaggio AI (`runAiAnalysis()`) analizza tutto il materiale disponibile e propone: stato del lead, obiezioni trasversali pertinenti, e una sintesi di analisi/strategia — sempre modificabile manualmente prima di procedere.

3.3 Generazione degli script

Alla generazione, `buildSystemPrompt()` compone il prompt di sistema inviato al modello AI assemblando, in quest'ordine di priorità dichiarata esplicitamente nel prompt stesso:

BLOCCO	ORIGINE	MODIFICABILE DALL'UTENTE
<code>HARD_COMPLIANCE_RULES</code>	Costante nel codice	No — sempre attiva, priorità massima
Obiettivo primario della chiamata	Costante nel codice	No
Profilo azienda	<code>COMPANY_PROFILE_DEFAULT</code> + eventuali modifiche salvate	Sì, da tab "Profilo Azienda"
Regole di compliance aggiuntive	Campo del profilo	Sì
Libreria script di riferimento	<code>SCRIPT_LIBRARY</code> (25 script)	No — vive solo nel codice

`buildUserPrompt()` aggiunge invece i dati specifici del lead corrente: stato selezionato, dati anagrafici/note, analisi/strategia, canali richiesti (telefono/WhatsApp/email), obiezioni da presidiare. Il modello risponde con un JSON contenente uno script per ciascun canale richiesto.

Il dettaglio delle regole di compliance (§9) è trattato separatamente per la sua rilevanza.

3.4 Stato del lead e storico

Dieci stati predefiniti (`LEAD_STATES`) mappano ciascuno a uno o più canali/script di default (es. "Trattativa persa — recupero" → telefono + email). Ogni lead lavorato viene salvato nello storico (`leadsHistory`), persistito via `db` / `localStorage`), consultabile, duplicabile come base per un nuovo lead, stampabile.

04 Trasferimento verso Suggerimenti Vendita

4.1 Payload e canale di trasporto

Il pulsante "Invia a Suggerimenti Vendita" (disponibile sia dopo una generazione appena fatta, sia riaprendo un lead dallo storico) costruisce un payload minimale — **solo** il canale telefono, mai email/WhatsApp né l'analisi interna, perché Suggerimenti Vendita gestisce esclusivamente telefonate:

STRUTTURA DEL PAYLOAD

lead

nome, segmento, stato_pipeline (mappato dallo stato lead)

canovaccio

titolo, testo (script telefono generato)

Il payload viaggia con lo stesso meccanismo del CRM: `window.postMessage` dalla pagina verso `console-content.js`, che lo scrive in `chrome.storage.local` (chiave `pendingLeadSuggerimentiVendita`) e notifica il service worker.

4.2 Apertura automatica dell'applicazione

Per far comparire l'app nativa direttamente dal browser, `Suggerimenti Vendita.app` registra uno schema URL personalizzato (`suggerimentivendita://`) tramite `CFBundleURLTypes` nel proprio `Info.plist`. La console apre un link invisibile con quello schema (non `window.location`, per non rischiare di navigare via dalla pagina se lo schema non fosse ancora registrato): macOS Launch Services instrada l'evento all'app, che lo riceve nel metodo `application_openURLs_` e porta la propria finestra in primo piano (avviandosi da sé se non era già in esecuzione).

NOTA DI ROBUSTEZZA

La registrazione dello schema presso Launch Services non è sempre automatica alla comparsa di una nuova chiave `CFBundleURLTypes`: l'eseguibile dell'app (`Contents/MacOS/avvia`) richiama esplicitamente `lsregister -f` su se stesso ad ogni avvio, in background, per auto-guarire eventuali registrazioni mancanti senza intervento manuale.

4.3 Segnalazione "lead in attesa"

Un piccolo server HTTP (porta `8767`), interno al processo del menu principale dell'app (non al server delle chiamate), espone `POST/DELETE /lead-in-attesa`: la console lo notifica al momento dell'invio, così il pulsante "Chiamata Gestione Lead" nel menu può accendersi con un bagliore visivo **prima ancora** che l'operatore avvii una telefonata. È un canale deliberatamente separato dal server delle chiamate (porta 8766), che parte solo alla prima telefonata della sessione — se dipendesse da quello, l'avviso non potrebbe mai comparire nel momento in cui serve di più.

Poiché l'app potrebbe impiegare qualche secondo ad avviarsi, la notifica viene ritentata più volte a distanza di 1,5s (fino a 8 tentativi) — dal content script della console, non dal service worker: i timer (`setTimeout`) in un service worker Manifest V3 non sono affidabili, perché Chrome può sospendere il processo tra un evento e l'altro senza rispettare i timer pendenti, mentre una pagina normale come la console resta viva per tutta la finestra di ritentativi.

05 Applicazione nativa Suggerimenti Vendita

5.1 Struttura del bundle

App macOS autosufficiente: `Contents/MacOS/avvia` (script bash, è il `CFBundleExecutable`) avvia — tramite PyObjC — una finestra Cocoa (`menu_finestra.py`) che carica `menu/index.html` in una `WKWebView` . Ogni pulsante del menu manda un messaggio JavaScript→Python (`WKScriptMessageHandler`), che rilancia lo stesso eseguibile con `--azione <nome>` per eseguire la singola funzione richiesta (le funzioni vere e proprie — setup, configurazione, avvio chiamata — restano tutte nello script bash, il menu grafico ne è solo un selettore).

DECISIONE ARCHITETTURALE

Il lancio del processo Python avviene con `exec` (non una sotto-chiamata): questo sostituisce il processo bash con quello Python **mantenendo lo stesso PID**, condizione necessaria perché macOS Launch Services riconosca correttamente l'app come "già in esecuzione" e instradi lì gli eventi successivi (es. l'apertura via schema URL, §4.2) invece di avviarne una seconda istanza.

5.2 Le tre modalità di chiamata

MODALITÀ	PARAMETRO URL	CANOVACCIO GUIDA	USO PREVISTO
Chiamata YesMobility	<code>avvio=""</code>	Mai mostrato	Sola gestione telefonata + obiezioni
Chiamata Gestione Lead	<code>avvio=gestione_lead</code>	Importato da Lead Rework Console	Chiamata su un lead specifico preparato
Rinforzo Facile Salire	<code>avvio=rinforzo_facile_salire</code>	Canovaccio fisso da file	Richiamo standard post-sopralluogo

Ognuna apre la stessa pagina (`overlay/index.html`) con un parametro `avvio` diverso in coda all'URL; è quel parametro a determinare, lato content script (§7), se e quale canovaccio va mostrato.

06 Motore di ascolto e suggerimento in tempo reale

6.1 Cattura audio

Il microfono viene catturato dal **browser** (`getUserMedia` dentro `overlay/index.html`), non dal processo nativo: un'app macOS non firmata con certificato sviluppatore non può ottenere in modo affidabile il permesso microfono di sistema, mentre un browser regolarmente firmato (Chrome) sì. I frame audio grezzi (PCM16 mono 16kHz) vengono inviati al server locale sulla stessa connessione WebSocket usata per ricevere i suggerimenti.

6.2 Pipeline di elaborazione

DAL PARLATO DEL CLIENTE AL SUGGERIMENTO A SCHERMO

1. Audio dal browser

PCM16 16kHz via WebSocket verso `server_suggerimenti.py`



2. Deepgram (STT streaming)

Trascrizione in tempo reale, si attende la trascrizione "finale" della frase



3. Motore di matching

`MotoreSuggerimenti.suggerisci()` — classificatore su embedding, confronta la frase con la libreria di script ammessi per la fase di chiamata corrente



4. Broadcast via WebSocket

Fase chiamata, testo suggerimento, alternativa, categoria trigger → a tutte le pagine overlay collegate

6.3 Tempi di reazione

Il tempo che intercorre tra la frase finale riconosciuta e la comparsa del suggerimento a schermo è mantenuto sotto i **300 millisecondi**: il modello di matching (la parte computazionalmente più pesante, imbarcata all'avvio del server) resta caricato in memoria per l'intera sessione di lavoro invece di essere ricaricato ad ogni chiamata — è il motivo per cui la prima telefonata della sessione richiede qualche secondo in più (caricamento iniziale) mentre le successive partono pressoché istantanee. Il matching viene eseguito in un executor separato dall'event loop asincrono, per non bloccare la gestione simultanea di audio e WebSocket.

6.4 Persistenza tra chiamate

Il processo server resta acceso tra una chiamata e l'altra (avviato alla prima, terminato solo alla chiusura dell'app): ogni nuova connessione WebSocket in modalità `--browser-audio` resetta la fase di chiamata e ricarica la libreria script (`motore.nuova_chiamata()`), così un aggiornamento alla libreria fatto tra due telefonate è visibile dalla chiamata successiva senza riavviare nulla.

07 Consegna del canovaccio guida in chiamata

Il canovaccio (lo script preparato in Lead Rework Console) è deliberatamente **disaccoppiato** dal motore di suggerimento in tempo reale (§6): non transita mai per il server Python, arriva alla pagina della chiamata esclusivamente tramite l'estensione browser.

Il content script `suggerimenti-vendita-content.js`, iniettato sulla pagina `http://localhost:8766/*`, legge `chrome.storage.local` (chiave `pendingLeadSuggerimentiVendita`) al caricamento della pagina e ad ogni notifica ricevuta dal service worker, ma **solo se** il parametro `avvio` nell'URL corrente è `gestione_lead` — nelle altre due modalità (§5.2) il dato resta intatto in storage, non viene consumato, per essere ancora disponibile la volta successiva che si apre davvero la modalità corretta.

EVOLUZIONE STORICA DEL MECCANISMO

Una prima versione faceva transitare il canovaccio anche via file (`lead-corrente.json`, letto dal server ad ogni connessione WebSocket) per aggirare le restrizioni TCC di macOS sull'accesso a `~/Documents` per app non firmate. Il meccanismo è stato rimosso: applicava il canovaccio a **qualsiasi** modalità di chiamata (in conflitto con la distinzione introdotta al §5.2) e, se un file residuo restava nel bundle da un test precedente, il suo contenuto aveva sempre precedenza — mostrando permanentemente uno script vecchio indipendentemente da nuovi lead inviati.

08 Miglioramento continuo della libreria

Ogni frase finale del cliente, insieme all'esito del matching (script scelto, fase rilevata), viene salvata in un log append-only (`logs/frasi_raccolte.jsonl`, una riga JSON per frase). La voce "Rivedi frasi raccolte" nel menu apre uno strumento di revisione (`gestione_frasi_raccolte.py`) per scorrere le frasi non ancora revisionate ed eventualmente trasformarle in nuovi esempi per la libreria di script — così le situazioni realmente incontrate al telefono, non previste in origine, arricchiscono nel tempo la capacità del motore di riconoscerle. La scrittura di nuovi script (anche manuale) può essere assistita da un secondo modulo AI dedicato (`genera_testo_script.py`).

09Regole di compliance e riservatezza

Un sottoinsieme delle regole che vincolano la generazione degli script è stato deliberatamente **spostato fuori dal profilo azienda modificabile** e reso costante nel codice (`HARD_COMPLIANCE_RULES` in `lead-rework-console.html`), iniettato per primo — con priorità dichiarata esplicita su tutto il resto del prompt, inclusa la libreria di script di riferimento — nel prompt di sistema, e ribadito una seconda volta in chiusura come controllo finale prima della risposta.

REGOLA	VINCOLO
Nessun contatto precedente	Ogni script si presenta come primo contatto assoluto, indipendentemente dallo stato interno del lead (es. "trattativa persa" è un nome di fase, non prova di una chiamata già fatta)
Nessun dato inventato	Vietato attribuire al cliente dichiarazioni, stati d'animo o dettagli non presenti esplicitamente nei dati forniti
Riservatezza dei dati interni	Informazioni note internamente (relazioni familiari, condizioni di salute, dettagli tecnici del sopralluogo, per chi è il prodotto) non vanno mai rivelate finché non è il cliente stesso a menzionarle nella chiamata in corso
Obiettivo di chiusura	Lo script deve puntare a un accordo con il cliente, non limitarsi a proporre un nuovo sopralluogo come esito della chiamata

Ogni regola include, nel prompt stesso, un esempio concreto di violazione realmente osservata e da non ripetere — tecnica adottata perché una regola astratta si è rivelata meno efficace, nella pratica, di un esempio negativo puntuale.

10Disposizione delle finestre sullo schermo

Le tre superfici (Lead Rework Console, menu Suggerimenti Vendita, pannello chiamata) occupano posizioni fisse in percentuale della larghezza dello schermo primario — 40% / 19% / 40% da sinistra a destra — calcolate dinamicamente ad ogni apertura, non in pixel fissi:

FINESTRA	MECCANISMO DI POSIZIONAMENTO
Lead Rework Console	Chrome in modalità app (<code>--app=</code> , <code>--window-position</code> , <code>--window-size</code>), geometria letta via <code>osascript</code> (bash) o <code>chrome.system.display.getInfo()</code> (estensione)
Menu Suggerimenti Vendita	<code>NSScreen.mainScreen().visibleFrame()</code> (PyObjC) — esclude nativamente barra menu e Dock
Pannello chiamata	Stesso meccanismo della Console (Chrome app-mode con geometria calcolata via <code>osascript</code>)

Nessuna delle tre finestre resta forzatamente in primo piano rispetto alle altre: la disposizione affiancata rende superfluo un livello di finestra prioritario, che in precedenza causava sovrapposizioni indesiderate.

11 Sintesi: il flusso end-to-end

CRM

Operatore clicca "Invia a Lead Rework Console" sulla scheda del lead



`chrome.storage.local`

Lead Rework Console

Stato, obiezioni e strategia suggeriti dall'AI → script generati (telefono/WhatsApp/email), nel rispetto delle regole di compliance



`chrome.storage.local + schema URL`

Suggerimenti Vendita — apertura

App nativa avviata/portata in primo piano, bagliore "lead in attesa" acceso sul pulsante corretto

↓ operatore clicca "Chiamata Gestione Lead"

Chiamata in corso

Canovaccio sempre visibile (da estensione) + audio ascoltato, trascritto e classificato in tempo reale (<300ms) → suggerimento sulle obiezioni



Dopo la chiamata

Fraasi raccolte disponibili per revisione → arricchimento della libreria per le chiamate successive